

Forsøk 1



Pågår

NESTE: Lever oppgave



Legg til kommentar

Ubegrenset antall forsøk tillatt

▼ Detaljer

Guidelines

To get credit for this exercise, you must:

- **Hand in the exercise on time—delayed hand ins will not receive any credit.**
- Hand in the Jupyter notebook file (*.ipynb) and the same file exported as PDF. If PDF export fails, hand in the IPython notebook. Without the PDF, I will not be able to give you feedback directly in the document.
- **You must include your names and exercise group number at the top of the notebook:**
 - **No name, no credit!**
 - **One point is deducted if the group number is missing.**
- Place the exercises in the order given below in the notebook.
- Begin each exercise with a second-level heading (##) reading "Exercise X".
- Make reasonable attempts at solving all exercises.

For an introduction to how to typeset mathematics in notebooks, see, e.g., the [LaTeX section](#)

(<https://patrickwalls.github.io/mathematicalpython/jupyter/latex/>) of [Mathematical Python](#)

(<https://patrickwalls.github.io/mathematicalpython/>). You are also encouraged to look at the Markdown and LaTeX code in the lecture notebooks.

Exercise 1 (4 points)

One metric for the productivity of scientists is the [h-index](https://en.wikipedia.org/wiki/H-index) (<https://en.wikipedia.org/wiki/H-index>). It is defined as follows: For each scientific publication by a scientist, count the number of times it has been cited by other scientists (there are several organisations collecting such statistics). A scientist has an h-index of n if her n -th least cited paper has been cited at least n times. For example, [Google Scholar currently reports for me an h-index of 25 at present](#)

(<https://scholar.google.com/citations?user=YR5QfVEAAAAJ&hl=en&oi=sra>), i.e., my 25th-least cited publication has been cited (at least) 25 times.

1. Give an algorithm that determines the h-index of the scientist based on a list of publications and number of citations.
2. Discuss the worst-case and average-case run-time of your algorithm.

Exercise 2 (3 points)

Modify the List-Insert algorithm for doubly linked lists so that it supports insertion not only at the head of the list, but after an arbitrary element. If the modified algorithm is called as List-Insert(x, y), then y shall be inserted immediately after x . When $x = \text{NIL}$ is passed, y shall be inserted at the head. Give your algorithm in pseudocode. Discuss the runtime of your algorithm, taking into account that x gives you direct access to the list element after which to insert.

Exercise 3 (4 points)

Write an algorithm Delete-Singly(k) that deletes the element with key k from a *singly* linked list. If k occurs multiple times, delete the first occurrence. If k is not in the list, the algorithm shall signal an error. Make sure that your algorithm works correctly also if k is the first or last item and for lists with a single item. Give your algorithm in pseudocode. Discuss the runtime of your algorithm.

Exercise 4 (4 points)

Consider the open addressing approach to hash tables (Part 3 of recorded Lecture 10).

1. In the lecture, I mentioned that deleting from a hash table using open addressing is difficult, because we need to avoid gaps in each probe sequence. As one solution, I suggested that when deleting the entry at position $h(k, j)$, we move all later entries in the same probe sequence, i.e., $h(k, j+1)$, $h(k, j+2)$..., forward to close the gap. Can you think of a more efficient way to close the gap, requiring less data movement? Give an algorithm for this in pseudo-code.
2. Consider the hashing function from Weiss given in the lecture

$$h(k, i) = (h'(k) + (-1)^i i^2) \bmod m$$

for a hash table with $m = 11$ slots and auxiliary hash function $h'(k) = k \bmod m$. We now insert the following key-value pairs in this order:

$(0, A)$ $(2, B)$ $(1, C)$ $(0, D)$ $(3, E)$ $(5, F)$ $(2, G)$ $(6, H)$

Build the hash table and illustrate the different probe sequences in a figure (you can draw by hand and include a photo).

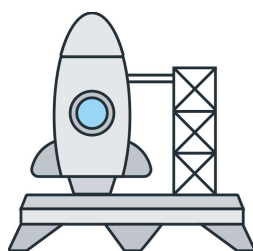
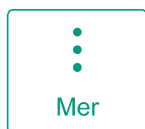
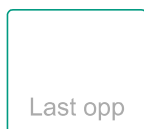
Exercise 5 (5 points)

You are given a list of intervals (a_k, b_k) . These could, e.g., represent times for which meeting rooms are booked. Create an algorithm that reports overlapping intervals. I.e., for $L = \{(2, 5), (6, 9), (7, 11), (10, 15), (12, 14), (13, 17), (19, 21)\}$ the algorithm should report that

- $(6, 9)$ overlaps with $(7, 11)$
- $(7, 11)$ overlaps with $(10, 15)$
- $(10, 15)$ overlaps with $(12, 14)$ and $(13, 17)$
- $(12, 14)$ overlaps with $(13, 17)$

Each overlap only needs to be reported once, i.e., report only that $(6, 9)$ overlaps with $(7, 11)$, but not that $(7, 11)$ overlaps with $(6, 9)$. Discuss the runtime of your algorithm.

Velg innsendelsestype



Velg en fil å laste opp
Tillatt fil: PDF, IPYNB

eller

Canvas-filer